

✦ Part 1 – Question Bank

Section A: 10-Mark Questions

1. Explain the step-by-step Getty and login process in a Linux system.
 2. Discuss the Syslog facility, detailing its core terms, facilities, and priority levels.
 3. Describe the architecture of systemd timer units, including their types and creation for user-level tasks.
 4. Elaborate on User Access Control in Linux, focusing on user categorization and the Pluggable Authentication Modules (PAM) framework.
-

Section B: 5-Mark Questions

5. Explain the structure of the `/etc` directory and the different types of configuration files it contains.
 6. Detail how to manage and synchronize system time using `date` and `timedatectl`.
 7. Compare the `at` command, `cron` jobs, and `systemd` timers for task scheduling.
 8. Describe the `logger` command, its parameters, and its role in system logging.
 9. Explain the concept of log rotation and identify important log locations in Linux.
 10. Outline the steps to create and manage a user-level systemd timer unit.
 11. Describe the four primary PAM management groups and their specific roles.
 12. Explain the numeric representation of file permissions and how to modify them using `chmod`.
 13. Detail the components and usage of cron expressions for repetitive task scheduling.
 14. Compare the advantages and disadvantages of using systemd timers versus traditional cron jobs.
-

Section C: 2-Mark Questions

15. Define the Getty process and its primary responsibility.
16. What is the purpose of the `/etc/motd` file during the login sequence?
17. Define a *selector* in the context of Syslog configuration.
18. Explain the basic usage of the `at` command for scheduling.
19. Distinguish between monotonic and realtime timers in `systemd`.
20. What is the function of the `systemd-getty-generator`?
21. What does the command `sudo timedatectl set-ntp true` achieve?
22. Identify the standard directory where individual service PAM configurations are stored.

★ Part 2 – Answer Key

10 Marks Questions

1. Explain the step-by-step Getty and Login process in a Linux system.

The login process is a sequential security operation managed by the Getty process to authenticate users and establish their interactive shell environment.

- **Getty Initiation:** The `init` process starts Getty after completing run-level execution and running commands in `/etc/rc.local`. This process is the foundation for handling all terminal connections and is now often managed by `systemd`.
- **Login Command:** Getty initiates the `login` command, which displays the `login:` prompt on the terminal screen. It waits for the user to provide a username and then prompts for a hidden password for security.
- **Credential Verification:** Getty verifies the entered credentials against the system authentication files `/etc/passwd` and `/etc/shadow`. If the password matches, the login process proceeds; otherwise, Getty terminates and restarts the prompt.
- **Property Gathering:** Upon successful login, Getty reads user properties such as UID, GID, home directory, and default shell from `/etc/passwd`. These details are then assigned to system variables like `$USER`, `$HOME`, and `$SHELL`.
- **Banner Display:** Before the user shell starts, Getty reads the `/etc/motd` file to display its contents as a banner. This "message of the day" provides important system-wide information to the user upon entry.
- **Global Shell Settings:** The process reads the `/etc/profile` file to apply system-wide shell settings and import aliases. This step ensures that every user shell begins with a standardized set of environmental variables.
- **Local Customization:** Getty reads user-specific files like `.bashrc` or `.bash_profile` located in the home directory to personalize the shell. This allows users to have customized aliases and paths unique to their own session.
- **Shell Identification:** The specific software to be launched as the user shell is determined by the `$SHELL` variable parsed from `/etc/passwd`. This ensures the user is placed in their preferred interface, such as Bash or Sh.
- **Process Handover:** Getty starts the user shell software to facilitate direct interaction between the user and the system. At this stage, control of the terminal session is handed over from Getty to the shell.
- **Prompt Presentation:** Finally, the shell presents the `PS1` prompt to the user, indicating readiness to execute commands. This marks the completion of the transition from a system process to a user-controlled session.

2. Discuss the Syslog facility, including its key terms, facilities, and priority levels.

Syslog is a host-configurable, uniform system logging facility that provides a centralized process for recording system and application events.

- **Syslog Facility Term:** "Facility" is an identifier used to describe the specific application or process that submitted a log message. Examples include the kernel, mail system, or FTP daemon, allowing for easy categorization of events.
- **Priority Term:** "Priority" serves as an indicator of the importance of a message, ranging from debugging info to critical system events. It defines the severity level that the system logger uses to decide how to handle the message.

- **Selector Term:** A "Selector" is a combination of one or more facilities and priority levels used to filter incoming events. When an event matches a selector, the system performs a predefined action based on the configuration.
- **Action Term:** "Action" defines what happens to a message once it matches a selector, such as writing it to a file or a console. Actions can also include sending messages to logged-in users or remote syslog servers.
- **Auth Facility:** The `auth` facility records activities related to requesting names and passwords, such as those from `getty`, `su`, or `login`. It is critical for monitoring security-related access attempts on the system.
- **Cron Facility:** The `cron` facility captures messages specifically from the cron system scheduler and its scheduled tasks. This allows administrators to track the success or failure of automated repetitive jobs.
- **Emergency Priority:** The `emerg` priority represents an emergency condition, such as an imminent system crash. These messages are usually broadcast to all users immediately to warn of critical system instability.
- **Error Priority:** The `err` priority is used for ordinary errors that occur during system or application operation. These are non-critical but indicate that something has gone wrong and requires attention from the admin.
- **Debug Priority:** The `debug` priority is used for messages generated while debugging programs to provide deep technical insight. These logs are usually ignored during normal operation to save disk space.
- **Selector Logic:** Syslog allows using an asterisk to indicate "all" for either a facility or a priority in a selector. For example, `*.debug` selects all debug messages from every facility on the system.

3. Elaborate on the architecture, types, and management of systemd timer units.

Systemd timer units offer a flexible and modern way to schedule tasks, integrating directly with system services and logging.

- **Dual Unit Architecture:** Scheduling a task via systemd requires two distinct unit types: a `.timer` file and a `.service` file. The timer file defines the schedule, while the service file defines the actual task to execute.
- **System-wide Timers:** System-wide timers run with root privileges and are typically stored in `/etc/systemd/system/` or `/usr/lib/systemd/system/`. These are used for administrative tasks that affect the entire operating system.
- **User-level Timers:** User-level timers run as regular users and are stored in the `~/.config/systemd/user/` directory. They only run when the specific user is logged in and do not require administrative access.
- **Monotonic Timers:** Monotonic timers schedule tasks relative to a specific event, such as system boot or the timer unit's activation. They use keywords like `OnBootSec` or `OnActiveSec` to define these relative time intervals.
- **Realtime Timers:** Realtime timers schedule events in absolute wall-clock terms using the `OnCalendar` keyword. This is similar to traditional cron schedules, allowing tasks to run at specific dates and times.
- **Persistent Execution:** By setting `Persistent=true` in a timer unit, systemd can catch up on missed jobs if the system was powered off. This ensures that critical tasks are not skipped due to system downtime.
- **Dependency Management:** Unlike traditional cron, systemd timers allow for dependency management between different services and units. This ensures that tasks only run if their required system conditions are met.

- **Logging Integration:** All execution history and errors for timer-activated services are logged via `journalctl`. This provides a centralized and detailed record for debugging and monitoring task performance.
- **Timer Status Monitoring:** Admins can view active timers and their next scheduled run times using the `systemctl list-timers` command. This report includes the last time the timer ran and how much time is left until the next execution.
- **Management Commands:** Managing timers involves using `systemctl` to enable, start, stop, or reload the units. A `daemon-reload` is specifically required after creating or modifying unit files to update `systemd`'s internal state.

4. Provide a detailed overview of User Access Control and the PAM framework in Linux.

User Access Control in Linux ensures system security by managing accounts, groups, and permissions through the flexible PAM framework.

- **User Categorization:** Linux divides users into three types: the Root user (UID 0) with full access, system users for services, and regular users for individuals. This hierarchy ensures that service processes have restricted access compared to administrators.
- **Account Management:** User accounts are managed via files like `/etc/passwd` for identity and `/etc/shadow` for secure password storage. Utilities like `useradd`, `usermod`, and `userdel` are used for account lifecycle management.
- **File Permissions:** Every file has three permission sets (Owner, Group, Others) which define who can read (r), write (w), or execute (x) it. These permissions are the first line of defense in restricting user access to data.
- **Permission Modification:** The `chmod` command is used to modify these access rights using either symbolic or numeric representations. For example, `chmod 755` provides full access to the owner and read/execute access to others.
- **PAM Definition:** Pluggable Authentication Modules (PAM) provide a centralized and modular framework for authenticating users across various system services. It allows admins to change security policies without modifying individual application code.
- **Auth Management Group:** The `auth` group in PAM manages the actual identification of the user, typically by requesting a password. It validates that the person attempting to log in is who they claim to be.
- **Account Management Group:** The `account` group verifies that the user account is valid and allowed to access the requested service. This checks for expired passwords or restricted access times for specific users.
- **Password Management Group:** The `password` group is responsible for the mechanism of updating the user's authentication token. It enforces rules like password complexity and prevents the reuse of old passwords.
- **Session Management Group:** The `session` group handles tasks that must be performed at the start and end of a user session. This includes mounting home directories or displaying the message of the day banner.
- **PAM Modularity:** PAM uses various modules like `pam_unix.so` for standard authentication or `pam_tally2.so` to limit failed login attempts. This modularity allows for the easy addition of Multi-Factor Authentication (MFA) or other security layers.

5-Marks Question

5. Describe the structure of the /etc directory and its configuration tables.

The `/etc` directory is the central location for system-wide configuration files and scripts that control how the Linux system and its services operate.

- **Configuration Tables:** This group includes files ending in ".tab" that act as tables for scheduling, mounting, and system initialization. They are static text files that the system reads to understand its operational parameters.
- **Crontab File:** Located at `/etc/crontab`, this table is used for scheduling repetitive system-wide tasks at specific intervals. It follows a specific syntax to define when and how commands should be executed automatically.
- **Fstab and Mtab:** `/etc/fstab` contains permanent file system mounting info, while `/etc/mtab` tracks the status of currently mounted devices. These tables are essential for the system to recognize and access storage partitions correctly.
- **Inittab File:** The `/etc/inittab` file defines the system's run-levels and specifies what the system should do at each stage of initialization. It determines the default state the computer enters after the booting process is complete.
- **Running Configurations:** The `/etc` directory also contains ".rc" files like `bashrc` and `rc.local` which set environments for users or services. These files force specific behaviors when a service starts or when a user logs into a shell.

6. Explain how to manage system time using date and timedatectl

Linux provides both the traditional `date` command and the modern `timedatectl` utility to view and modify system time and synchronization settings.

- **Viewing Time:** The `date` command is used to quickly check the current system time in the terminal. For more detailed settings including timezone and NTP status, the `timedatectl` command is used.
- **Manual Date Setting:** Users with root privileges can set the date manually using the format `sudo date MMDDhhmmYYYY`. This is useful for temporary adjustments but does not provide long-term network synchronization.
- **Timedatectl Manual Setting:** The `sudo timedatectl set-time "YYYY-MM-DD HH:MM:SS"` command allows for precise manual time adjustment via `systemd`. This utility is the modern standard for changing system clock settings.
- **NTP Synchronization:** Network Time Protocol (NTP) can be enabled using `sudo timedatectl set-ntp true` to keep the clock accurate via the internet. This ensures the system clock remains synchronized with global time servers automatically.
- **Disabling Synchronization:** If a manual time override is needed, automatic synchronization can be turned off with `sudo timedatectl set-ntp false`. This prevents the system from overwriting manual changes with network-provided time.

7. Compare the at command, cron jobs, and systemd timers for task scheduling.

Choosing the right scheduling tool depends on whether the task is a one-time event, a simple repetition, or a complex service-dependent job.

Feature	at Command	cron Jobs	systemd Timers
Task Type	One-time execution in the future.	Repetitive tasks at fixed intervals.	Repetitive tasks integrated with services.
Time Flexibility	Precise, any specific time.	Fixed schedules using crontab syntax.	Advanced time and monotonic expressions.

Logging	Minimal logging available.	Limited logging via syslog.	Detailed logging via journalctl.
Dependencies	No dependency management.	No control over task dependencies.	Can set service and unit dependencies.
Missed Jobs	Task is missed if system is down.	Jobs are skipped if system is off.	Can catch up with <code>Persistent=true</code> .

8. Detail the parameters and usage of the logger command.

The `logger` command is a shell interface to the `syslogd` daemon, allowing users to add manual entries to system log files from the command line.

- **Message Parameter:** The string arguments provided to `logger` are concatenated and added as the log message entry. This allows for the manual recording of script outputs or administrative notes into the system logs.
- **Priority Flag (-p):** This parameter allows users to specify the priority of the log entry as a facility.priority pair. By default, entries are logged with the `user.notice` priority if this flag is omitted.
- **File Input Flag (-f):** The `-f filename` option allows the `logger` command to use the entire contents of a specified file as the log message. This is extremely useful for logging large batches of data or report summaries.
- **Tagging Flag (-t):** The `-t tag` parameter marks each line of the log entry with a specific identifying tag. This makes it much easier to filter and find specific manual entries later using search tools.
- **PID Logging (-i):** Using the `-i` flag ensures that the process ID of the logger process is recorded along with the log message. This provides an extra layer of traceability for auditing which process generated the log.

9. Discuss Log Rotation concepts and important system log locations.

Log management involves both the storage of logs in standardized directories and the use of rotation tools to manage disk space consumption.

- **Log Rotation Purpose:** Log files grow rapidly and can exhaust disk space if not managed. Rotation tools like `logrotate` or `newsyslog` are used to archive old logs and start new ones periodically.
- **Rotation Mechanism:** These tools are typically triggered by the cron daemon at frequent intervals to compress and delete old log files. This ensures that only a manageable amount of recent log history is kept on the active disk.
- **Standard Log Directory:** Most system applications and services in Linux create their log files within the `/var/log` directory and its sub-directories. This provides a centralized location for administrators to monitor all system activity.
- **Service-Specific Logs:** Key services have dedicated directories, such as `/var/log/httpd` for web servers and `/var/log/samba` for file sharing. Other services like `mail` and `cron` also store their logs directly in the main `/var/log/` folder.
- **Database Logs:** Database applications like MySQL also follow this convention, typically storing their operational logs in `/var/log/mysql`. Centralizing these logs facilitates easier backups and security auditing across all installed applications.

10. Outline the steps to create and manage a user-level systemd timer unit.

User-level timers allow individuals to schedule personal tasks without requiring root access by using systemd units stored in their home directory.

- **Directory Creation:** First, the user must create the configuration directory at `~/.config/systemd/user/` if it does not already exist. This is where both the service and timer unit files must reside.
- **Service Unit Creation:** A `.service` file is created to define the actual task, specifying the command to execute in the `ExecStart` field. This file describes what the systemd unit will actually do when triggered.
- **Timer Unit Creation:** A `.timer` file is then created to define the schedule, using either monotonic or realtime `OnCalendar` expressions. It must also specify the target service it is intended to trigger.
- **Daemon Reload:** After saving the files, the user must run `systemctl --user daemon-reload` to make systemd aware of the new configurations. This step is mandatory for any changes to be recognized by the system.
- **Enabling and Starting:** Finally, the timer is activated using `systemctl --user enable --now mytask.timer`. This ensures the timer starts immediately and persists through subsequent user logins.

11. Describe the four primary PAM management groups and their specific functions.

PAM organizes its authentication tasks into four distinct management groups that handle different aspects of user session security.

- **Auth Group:** This group focuses on the primary authentication of the user by verifying their identity through methods like passwords or biometrics. It ensures that the person is a valid system user.
- **Account Group:** The account group performs non-authentication verification, such as checking if an account has expired or if the user is allowed access at this specific time. It validates the status of the user account itself.
- **Password Group:** This group is dedicated to managing password updates and enforcing strength policies. It provides the mechanisms necessary for users to change their authentication tokens securely.
- **Session Group:** The session group manages the environment before and after a user is granted access. This includes tasks like mounting a user's home directory or logging the start and end times of the session.
- **Module Integration:** Each of these groups can utilize various shared modules, such as `pam_unix.so` for standard password checks or `pam_motd.so` for displaying banners. This allows for highly customized security policies.

12. Explain Linux user categorization and the numeric representation of file permissions.

Linux utilizes a hierarchical user structure and a mathematical permission system to enforce data security and system integrity.

- **Root User:** The root user has a UID of 0 and possesses full, unrestricted access to the entire system. This account is intended for administrative tasks and can override any file permissions.
- **System and Regular Users:** System users are created for service processes to limit their scope, while regular users are created for human individuals using the system. This separation prevents a compromised service from gaining full system control.
- **Permission Types:** Permissions are categorized into Read (r), Write (w), and Execute (x). These define the specific actions a user can perform on a file or directory.

- **Numeric Representation:** Each permission is assigned a numeric value: Read is 4, Write is 2, and Execute is 1. By adding these numbers, a single digit can represent a full set of permissions for a user.
- **Calculating Permission Sets:** A value of 7 (4+2+1) represents full rwx access, while 5 (4+1) represents read and execute only. These digits are grouped for Owner, Group, and Others, such as in the command `chmod 755`.

2-Marks Question

13. Define the role of the Getty process in terminal connections.

Getty is the fundamental process in Linux responsible for establishing communication between the system and terminal devices.

- **Terminal Initialization:** Its primary role is to initialize the terminal (TTY) and present the initial login prompt to users.
- **Session Startup:** Once user input is received, Getty hands control to the login program to begin the authentication sequence.

14. What is the purpose of the /etc/motd file during login?

The `/etc/motd` file is a standard configuration file used to communicate information to users upon a successful login.

- **Banner Message:** It contains the "message of the day," which Getty reads and displays as a banner just before starting the user shell.
- **Information Sharing:** It is typically used by administrators to broadcast system-wide news, maintenance schedules, or legal warnings to all users.

15. Define a "Selector" in the context of Syslog configuration.

In Syslog, a selector is a filtering mechanism that determines which incoming log messages should be processed.

- **Logic Pairing:** It consists of a combination of one or more facilities and priority levels, such as `kern.debug`.
- **Action Trigger:** When an incoming system event matches a selector's criteria, it triggers a specific action, like writing the message to a log file.

16. State the primary use case for the at command.

The `at` command is a specialized scheduling utility designed for non-repetitive task automation.

- **One-Time Execution:** Its primary use case is to schedule a task or script to run exactly once at a specific time in the future.
- **Future Automation:** Unlike cron, it is ideal for tasks that do not need to repeat, such as a one-off system update or a scheduled file transfer.

17. Differentiate between Monotonic and Realtime timers in systemd.

Systemd timers are categorized based on how they calculate the time for task execution.

- **Monotonic Timers:** These schedule tasks relative to a system event, such as a certain number of minutes after the system boots.
- **Realtime Timers:** These schedule tasks in absolute "wall-clock" terms, such as running a script every Monday at 10:00 AM.

18. Identify the standard directory for system application log files.

Linux follows a standardized directory structure for storing log data to make system monitoring easier for administrators.

- **Central Location:** The standard directory for almost all system and application log files is `/var/log`.
- **Organizational Structure:** Most complex applications, such as web servers or databases, use sub-directories within `/var/log` to organize their specific log data.

19. Explain the function of the `-p` parameter in the `logger` command.

The `-p` parameter is a critical flag for the `logger` command that allows for precise control over log severity.

- **Priority Specification:** It allows the user to manually set the priority and facility for the log entry, such as `user.warn`.
- **Default Overriding:** If this parameter is not used, the command defaults to the `user.notice` priority for all manual entries.

20. What is achieved by the command `sudo timedatectl set-ntp true`?

This command is used to manage the system's time synchronization settings via the `systemd` utility.

- **Network Sync:** It enables automatic time synchronization over the network using the Network Time Protocol (NTP).
- **Clock Maintenance:** This ensures the system clock remains accurate by periodically communicating with global time servers on the internet.

21. Briefly define a `systemd` service unit vs. a timer unit.

Scheduling in `systemd` relies on the interaction between two different types of configuration files.

- **Service Unit:** This unit defines the "what"—the specific task, script, or command that needs to be executed.
- **Timer Unit:** This unit defines the "when"—it contains the schedule and specifies which service unit should be triggered at that time.

22. Where are the individual service configuration files for PAM located?

PAM uses a centralized directory to store the unique authentication policies for different system applications.

- **Config Directory:** The individual configuration files for services like SSH, sudo, or login are located in `/etc/pam.d/`.
- **Service Isolation:** Each file within this directory is named after the service it configures, allowing for independent security policies for every application.